# Qwen3-VL-2B-Instruct: Complete Tuning & Findings Log

Full record of every configuration change, fix, benchmark, and finding made against Qwen3-VL-2B-Instruct in the VisionPsy-Nano PAN/ID extraction Android app, in roughly chronological order. Model: `qwen3vl-2b-instruct-q4_k_m.gguf` (LM) + `mmproj-qwen3vl-2b-instruct-q8_0.gguf` (vision projector), run via llama.cpp's `llama-mtmd-cli` (one-shot) or `llama-server` (persistent) on-device.

**Test device:** Samsung Galaxy M35 5G (SM-M356B), Exynos 1380 (5nm, octa-core big.LITTLE: 4× Cortex-A78 @2.4GHz + 4× Cortex-A55 @2.0GHz), Mali-G68 MP5 GPU (Valhall, 5 shader cores, Vulkan 1.2), ~5.3GB usable RAM. A second, higher-RAM (8GB) unit of the same SoC family was also used for early testing.

---

## Current status (as of 2026-09-29) — where we stopped

A quick-reference snapshot, separate from the chronological log below. Read this first if picking the Qwen3-VL work back up.

**Locked in, not under active investigation:**

- Thread count 4, `-Cr 4-7`/`--cpu-strict` core-pinning flags set (see the §1 correction — the flags are set but confirmed **not actually working**), `-fa on`, mmap on / mlock off, NEON+dotprod present, AssetManager ruled out as a bottleneck, GPU toggle off by default (Vulkan confirmed a net loss on this hardware), `applyDynSizeEnv` correctly gated to PaddleOCR-VL only.
- `pan_full` image-token cap: manual toggle, default **256**, confirmed to fix every real accuracy error found at 128 on the 5 ground-truthed clean cards.

**The one real open lead, not yet acted on:**

- The OpenMP core-pinning gap (§1). This is the single highest-confidence remaining latency lever for Qwen specifically — unlike the MiniCPM-V slice-count problem (a structural, model-architecture cost), this is a **plain bug**: the pinning flags are already there, already intended to work, and silently don't. Two fix paths are written up and ready to try (swap to `build-android-opt`, or add `OMP_PLACES`/`OMP_PROC_BIND` env vars) — neither has been implemented or measured yet. This is the natural next step for Qwen if latency work resumes.

**Testing gaps, not blockers, just incomplete:**

- Blurred-image accuracy at cap 128: zero real data (one attempt hung and was killed before the manual toggle existed; never retried since).
- 3 of the 5 ground-truthed clean cards never tested at cap 128 (ANISH SANJIVA SHETTY, MANIKANDAN SRIDHARAN, PRAYAGRAJ BEHERA).
- Every full-card doc type other than `pan_full`/`pan_crop` (aadhaar, dl, passport, tt_*) is still sitting at the older, pre-2026-09-24 default of 1024 image tokens — never put through the same on-device rigor `pan_full` got. This is the biggest *unknown* in the whole tuning history: it might be fine, or it might be carrying the same kind of "128 was marginal" surprise `pan_full` had, and nobody's checked.

**Why work moved away from Qwen mid-session:** the 128-vs-256 investigation and the OpenMP-gap discovery both happened while validating Qwen, but attention then shifted to bringing up MiniCPM-V 4.6 as a second model (see the separate `HANDOVER.md` in this repo for that thread) — not because Qwen's open items were resolved, but because a second model's correctness bugs (thinking-mode runaway decode, a server-mode chat-template gap) were more urgent to fix first. Qwen's own open item (§ above) is still exactly where it was left.

---

## 1. CPU threading & core placement

- **Thread count:** `-t`/`-tb` set from a UI slider, default **4** — chosen because it exactly matches the phone's 4 fast A78 cores. Reasoning: llama.cpp's threaded matmul has a per-layer barrier, so using 8 threads would force fast A78 threads to wait on slow A55 threads every layer; 4 is the right number for this big.LITTLE layout, not just a CPU% mitigation.
- **Core pinning (added 2026-09-24):** `-Cr 4-7 -Crb 4-7 --cpu-strict 1 --cpu-strict-batch 1` — pins compute threads to cores 4-7 (the fast A78 cores on this Exynos 1380; cores 0-3 are the slow A55s, confirmed via `/proc/cpuinfo` CPU part IDs `0xd41` vs `0xd05` on two separate M35 5G units). Without this, `-t`/`-tb` only sets a thread *count* — the OS scheduler can still migrate those threads onto slow cores, which was the leading theory for 8-16 tok/s prefill-rate swings observed across otherwise-identical requests.
- **`-fa on` (forced flash-attention):** the `auto` default was leaving flash attention off in practice; forcing it on measurably helped prefill throughput in on-device testing 2026-09-24.

### ⚠️ Correction (2026-09-28): the core-pinning fix does not actually work

Discovered while investigating latency variance: the binary currently deployed to `jniLibs` (built from `build-android-vulkan`) was compiled with **`GGML_OPENMP=ON`**. In `ggml/src/ggml-cpu/ggml-cpu.c`'s `ggml_threadpool_new_impl()`, the cpumask from `-Cr`/`-Crb`/`--cpu-strict` is only ever *applied* via `ggml_thread_apply_affinity()` inside the `#else // GGML_USE_OPENMP` branch (the pthread-based worker pool). Under `#ifdef GGML_USE_OPENMP`, the mask is computed but **never applied** — OpenMP spawns its own runtime threads, invisible to that code path. No `OMP_PLACES`/`OMP_PROC_BIND` env vars are set anywhere in the app to compensate. Net effect: **the pinning flags have been silently a no-op the whole time on the deployed binary**, and threads can still land on slow cores at random — the most likely real explanation for the prefill-rate swings the flags were originally added to fix.

**Not yet fixed.** Two options identified, neither applied yet:

1. Swap `jniLibs` to the `build-android-opt` binary (`GGML_OPENMP=OFF`, real pthread affinity works) — loses Vulkan/GPU capability, which is fine since GPU offload is already a net loss on this hardware (see §3).
2. Keep the Vulkan-capable binary, add `OMP_PLACES`/`OMP_PROC_BIND` env vars via `ProcessBuilder.environment()` (same pattern as the existing image-token-cap env var) to control OpenMP's own thread affinity instead.

Expected impact if fixed: latency should cluster near the low end of already-observed ranges instead of swinging to the high end — inference from variance data, not yet a measured before/after.

---

## 2. Image-token-cap tuning for `pan_full` — the full 128 vs 256 saga

## Original validation (2026-09-24)

128 image tokens for `pan_full`, tested against one clean card (`pan_sample.png`, 2127×1282, 10.8MP) with a lean prompt (no shared system prompt — grammar already enforces its rules) plus the core-pinning + flash-attention changes above: **cold, uncached, single request: 33.55s, all 5 fields exact match**, including DOB (the field most likely to break first).

A full 128→2048 sweep on that *same single image* showed:

| Cap | Latency | Accuracy |
|---|---|---|
| 128 | 33.55s | 5/5 |
| 256 | 48.3s | 5/5 |
| 512 | 87.1s | 5/5 |
| 1024 | 169.9s | 5/5 |

Accuracy held at 5/5 all the way through 1024 on this one image — latency climbed monotonically with no accuracy cliff. This *contradicted* the original theory (that Qwen's own "minimum 1024 image tokens for grounding tasks" warning meant 1024 was a hard floor) on this specific document. Caveat noted at the time: this was **one card only**, not yet cross-checked against multiple different cards.

## Blur-adaptive cap (built 2026-09-24, later abandoned)

Built a per-image Laplacian-variance blur-detection switch: downscale to 300px grayscale, compute the population variance of a 4-neighbor Laplacian response (a standard cheap sharpness metric), switch cap to 256 if variance was below a calibrated threshold (sharp = more headroom already, so 128 was fine; blurred = needs the extra tokens to recover detail a sharpness loss destroys).

**Calibration data** (5 real ground-truthed PAN cards + their Gaussian-blurred, radius-2.2 copies):

- All 5 clean cards scored **765.9 – 2437.5** variance.
- All 5 blurred copies scored **132.9 – 350.5** variance.
- Clean gap, no overlap — threshold set at **500** (roughly the midpoint, margin both ways).

## The finding that killed the blur-adaptive approach

On-device testing (2026-09-24) found that **256 also fixes misreads found at 128 on CLEAN, non-blurred cards** — meaning 128 was marginal for dense print regardless of blur, not a blur-specific problem:

- **SOHRAAB DANISH** card at cap 128: name and parent_name both garbled ("SOHRAB"/"AZFAL"-shaped errors), reproduced twice (Run 147, Run 149). At cap 256 (Run 153, Run 157): both fields came back correct, reproduced twice.
- **SAROJINI M** card: cap 128 misread a PAN-number digit.

## Ground-truth correction (2026-09-28)

While investigating the SAROJINI M mismatch between a cap-128 run and a cap-256 run, zoomed into the actual card image directly. **The real PAN number is `LUQPS8083N`, not `LUQPS8003N`** as originally transcribed. This means:

- The cap-128 run (which read `LUQPS8003N`, matching the *wrong* old transcription) had been mis-graded as correct. It was actually wrong (misread 8→0).
- The cap-256 run (which read `LUQPS8083N`) was actually correct.

### Corrected final comparison (clean images, cap 128 vs 256)

| Card | 128 | 256 |
|---|---|---|
| SOHRAAB DANISH | 3/5 (name, parent_name wrong) | **5/5** (both fixed, reproduced twice) |
| UDAYRAJ SINGH | 5/5 | 5/5 (unchanged) |
| SAMAD | PAN # correct; rest unverified | same output |
| MANIKANDAN SRIDHARAN | not tested at 128 | 4.5/5 (parent_name right letters, wrong case — cosmetic) |
| SAROJINI M | 4/5 (pan_number wrong — corrected grading) | **5/5** (pan_number now correct) |

**256 fixed every real error found at 128** on every card with solid ground truth, with only one cosmetic (case-only) blemish. Cost: latency goes from ~32s average (128) to ~43s average (256) on clean images.

### Final decision: manual toggle, not auto-detection

The blur-detection code (`laplacianVariance()`, `BLUR_VARIANCE_THRESHOLD`) was **deleted entirely**, not kept as a fallback "auto" option — the variable it was detecting for (blur) turned out not to be the actual driver of the accuracy gap. Replaced with a simple UI switch, `switchPanFullHighCap` in `activity_main.xml`, read directly in `imageTokenCapFor()`:

```
private fun imageTokenCapFor(docType: String): Int = when {
    singleFieldCropDocTypes.contains(docType) -> 256
    docType == "pan_full" -> if (binding.switchPanFullHighCap.isChecked)
256 else 128
    else -> 1024
}
```

Default: **ON (256)**. Off drops back to 128 for when speed matters more than the ~10-15s accuracy premium.

### Testing still incomplete as of the last check

- Blurred images: **zero** successful 128-cap results across all 5 ground-truthed blurred cards (one early attempt hung 110s+ and had to be killed, before the toggle existed).
- Clean images never tested at 128: ANISH SANJIVA SHETTY (never run at any cap), MANIKANDAN SRIDHARAN (only run at 256), PRAYAGRAJ BEHERA (only its blurred version was tested, at 256).

---

## 3. GPU (Vulkan) offload — built, tested, rejected

- Built `build-android-vulkan` with `-DGGML_VULKAN=ON` via a fully local, portable toolchain (own cmake+ninja, NDK's bundled `glslc`, a hand-written SPIRV-Headers CMake package since no host compiler existed for the real one, a statically linked musl host toolchain for the `vulkan-shaders-gen`/`llama-ui-embed` host tools, downloaded Vulkan-Hpp C++ headers since the NDK only ships the C API). This build auto-enabled `GGML_OPENMP=ON` (the old CPU-only build had it off) and needed `libomp.so` bundled into `jniLibs/` to avoid a `CANNOT LINK EXECUTABLE` error — this is the same binary later found to break CPU core-pinning (§1).
- **Result: GPU offload is a net loss, not a win.** Tested `-ngl 99 --no-mmproj-offload` (LM on Vulkan GPU, vision encoder on CPU) on a PAN card image: ran 117.5+ seconds (still running when checked) with GPU pegged at 100% (clock 949MHz, not thermally throttled — genuinely maxed out) and CPU at 0%. Far slower than the CPU-only baseline for the same lightweight workload.
- **Root cause:** build log showed `GL_KHR_cooperative_matrix not supported by glslc` — the Mali-G68 MP5 lacks the fast cooperative-matrix Vulkan extension, so llama.cpp's Vulkan matmul falls back to a slow generic compute-shader path.
- **Recommendation:** keep the GPU toggle **OFF** as default for this device family. This corrects an earlier prediction (before measurement) that Vulkan would give a "modest" speedup — actual measurement showed a regression instead. The cooperative-matrix gap is a hardware/driver limitation, not a config bug, so it won't be fixed by flag tweaking.

---

## 4. Prompt / chat-template handling

- Qwen does **not** need `--jinja` — it works through the CLI's built-in default ChatML-style formatting (`-sys`/`-p`), unlike PaddleOCR-VL and MiniCPM-V, which both ship real custom Jinja chat templates requiring `--jinja`.
- **Bug found and fixed:** `applyDynSizeEnv()` (PaddleOCR-VL's `MTMD_CROP_THRESHOLD_PIXELS`/`MTMD_FULLIMAGE_MAX_PIXELS` env vars) was originally called **unconditionally for every model**, which silently nullified Qwen's own image-token cap for every "full image" request — confirmed on-device: `pan_full` produced byte-identical prompt token counts (1398) at cap 1024, 512, and 256 until this was gated. Fixed by scoping the call to PaddleOCR-VL only.
- Qwen's image-token cap is applied through llama.cpp's own upstream `--image-max-tokens` equivalent (`LLAMA_ARG_IMAGE_MAX_TOKENS` env var → `custom_image_max_tokens` → `set_limit_image_tokens()` in `clip.cpp`), baked in at model-load time — "only used by vision models with dynamic resolution" per its own `--help` text. Confirmed a no-op for VisionPsy (fixed-tile idefics3) and PaddleOCR-VL (reads GGUF metadata directly, never touches `custom_image_max_tokens`).
- No min-tokens floor forced for crops — Qwen's own stock 8-token floor is fine, since a real single-field crop from this phone's camera naturally sits around 70-95 tokens already.

---

## 5. Correctness checks performed on the deployed build (2026-09-24 to 2026-09-28)

All confirmed directly from source/build config, not assumption:

| Check | Result |
|---|---|
| `use_mmap` | **Enabled** (default; `--no-mmap` never passed anywhere) |

| Check | Result |
|---|---|
| `use_mlock` | **Disabled** (default; `--mlock` never passed anywhere) |
| NEON / ARMv8 vector extensions | **Present** — `GGML_CPU_ARM_ARCH=armv8.2-a+dotprod+fp16`, `HAVE_DOTPROD=1` confirmed in `CMakeCache.txt` for both `build-android-opt` and `build-android-vulkan`. NEON is baseline-mandatory on arm64-v8a anyway; this build goes further with dot-product + fp16 extensions. |
| AssetManager as a model-loading bottleneck | **Ruled out.** `AssetManager` is used exactly once in the app, only for small prompt/schema text files (100KB total `assets/` folder). Model `.gguf` files (1GB+) are **not** bundled as APK assets — they live as plain files under `getExternalFilesDir()/models/`, opened directly by the native process via a normal filesystem path, completely bypassing `AssetManager` and any APK-compression/streaming constraints. |
| OpenMP thread contention / core pinning | **Broken** — see §1. The one open, unresolved item from this list. |

---

## 6. Server mode / persistent server

- Toggle: "Persistent server mode (experimental)". Off = reloads the full model every run (current baseline). On = keeps the model resident across runs and applies a vision-tile latency fix (~14-16s/field vs ~50-90s), but the **first** run after changing model/threads/GPU pays one restart cost.
- `ensureServerRunning()` only reuses a warm server when `modelPath`, `mmprojPath`, `threads`, `useGpu`, and `imageTokenCap` **all** match the previous request — switching doc types (which can change the effective image-token cap) or reinstalling the app both force a fresh restart, same mechanism.
- Unlike MiniCPM-V (a hybrid attention/SSM architecture that cannot reuse cached prompt state across requests — llama-server logs `forcing full prompt re-processing due to lack of cache data (likely due to SWA or hybrid/recurrent memory)` every time), **Qwen's standard transformer architecture reuses prompt cache/context checkpoints normally** across warm requests — a real, structural latency advantage Qwen has over MiniCPM-V for repeated same-image or similar-prefix requests in server mode.

---

## 7. Grammar-constrained decoding & known limitations

- Structured extraction uses `-jf`/`json_schema` grammar-constrained decoding — the GBNF grammar generated from each doc type's JSON schema forces syntactically valid JSON output.
- **Known llama.cpp grammar limitation:** "contains X" style regex patterns (`.*X.*`) in a `--json-schema` are **not reliably enforced** by the grammar compiler. Workaround: use `maxLength`/anchored patterns in the schema instead, and catch any remaining semantic issues (e.g. a value that's technically schema-valid but semantically wrong) in Kotlin post-processing rather than relying on the grammar alone.
- **Prompt-copying behavior:** the model tends to copy prompt text that is "value-shaped" (looks like an example answer) instead of actually reading the image, when the prompt includes few-shot-style

formatting hints. Mitigation: avoid few-shot examples in prompts; lean on grammar constraints to force the right *shape* of output instead of showing the model example *values*.
- **The one rule grammar can't enforce:** don't guess on an unreadable field, use `null` instead — this has to be handled by prompt instruction + Kotlin-side validation, not the grammar itself, since a grammar can force valid JSON *shape* but can't judge whether a given string is a real read or a hallucinated guess.

---

## 8. Related fine-tune / doc-type work (context, not Qwen-specific tuning)

- **Stage1 fine-tune** (`pan_stage1`/`aadhaar_stage1` doc types): a format-fix, not an accuracy-fix, fine-tuned checkpoint for PAN/Aadhaar only. Uses `maxTokens = 1024`, skips the JSON-grammar contract (relies on the model's own trained formatting) and skips the shared system prompt (the training data already embeds a self-contained instruction block with its own schema-as-prose). Fully reversible to stock behavior for every other doc type.
- **T&T document types** (`tt_passport`/`tt_dl`/`tt_id`) are early and unverified compared to the hardened `pan`/`aadhaar`/`dl` types — flagged as needing real test images before further tuning, not yet part of the validated tuning history above.
- **Aadhaar address fabrication catalog:** a separate, Aadhaar-specific set of confirmed model fabrication patterns (duplicate address text, name/DOB leaking into the address field, enrolment-number leaking in, pincode mismatches) and their corresponding Kotlin-side validation checks — included here for context since it's part of the same "know the model's real failure modes, don't just trust grammar-valid output" theme as §7, but it's Aadhaar-specific, not Qwen image-token-cap tuning.

---

## 9. Summary: what's confirmed-good vs still-open

**Confirmed working / validated:**

- Thread count (4), flash-attention forced on, NEON/dotprod compiled in, mmap on, mlock off, AssetManager not a bottleneck, image-token-cap 256 default for `pan_full` with a manual 128/256 override toggle, GPU toggle correctly defaulted off, `applyDynSizeEnv` correctly gated to PaddleOCR-VL only.

**Still open / not yet fixed:**

- **Core pinning is a confirmed no-op** on the deployed OpenMP-enabled binary — the single highest-priority open item, since it's the most likely explanation for remaining run-to-run latency variance. Two fix paths identified (swap to `build-android-opt`, or add `OMP_PLACES`/`OMP_PROC_BIND` env vars), neither applied yet.
- Blurred-image accuracy at cap 128: zero data points, one hung/killed attempt.
- Three clean cards never tested at cap 128 (ANISH SANJIVA SHETTY, MANIKANDAN SRIDHARAN, PRAYAGRAJ BEHERA).
- Non-`pan_full` full-card doc types (aadhaar, dl, passport, tt_*) still sit at the older, more conservative 1024-token cap, carried over from before any on-device validation — never re-tested against the same rigor `pan_full` got.